

Ex. Nos. 1	Installation and Configuration of Kali Linux in Oracle VirtualBox
Date :	

Aim

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Kali Linux is a Debian-based Linux distribution developed for penetration testing, digital forensics, and cybersecurity analysis.

Installing Kali Linux inside Oracle VirtualBox provides a safe and isolated environment to practice security experiments without affecting the host system.

VirtualBox is an open-source virtualization tool that allows running multiple operating systems simultaneously on a single physical machine.

Procedure

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

- Download Oracle VirtualBox from the official website.
- Run the installer and follow default installation steps.
- Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

- Open Oracle VirtualBox.

- Click New.
- Enter:
 - Name: Kali Linux
 - Type: Linux
 - Version: Debian (64-bit)
- Click Next.

Step 5: Allocate Memory and CPU

1. Memory Size: 2048 MB or higher
2. Processor: 2 CPUs
3. Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

Step 7: Attach Kali Linux ISO

1. Select the created VM → Settings.
2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

4. Start the virtual machine.
5. Select Graphical Install.
6. Choose:
 - a. Language
 - b. Location
 - c. Keyboard layout
7. Configure network automatically.
8. Set:
 - a. Username
 - b. Password
9. Choose:
 - a. Guided – Use entire disk
 - b. All files in one partition
10. Confirm disk changes and continue installation.
11. After completion, reboot the system.

Result

Kali Linux was successfully installed and configured in Oracle VirtualBox. The virtual machine is ready for performing network security and ethical hacking experiments.

Ex. Nos. 2	Encryption – Crypto 101 using TryHackMe Platform
Date :	

Aim

To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Theory (Brief)

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.

Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

Steps to Perform the Experiment

Step 1: Login to TryHackMe

3. Open browser and go to: <https://tryhackme.com>
4. Login using your registered credentials.

Step 2: Open Crypto 101 Room

6. In the search bar, type **Crypto 101**
7. Click on the room named **Crypto 101**
8. Click **Join Room**

Step 3: Start the Machine (if required)

1. Click **Start Machine / Start AttackBox**
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

6. Copy the given Base64 string from the question.

7. Open the TryHackMe terminal.

8. Give the command in the terminal prompt:

echo "Q3J5cHRvZ3JhcGh5" | base64 -d

12. Change the Input and get different outputs.

13. The output will be the decoded plain text.

14. Enter the decoded text into the answer box.

To Encode Base64

echo "cryptography" | base64

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

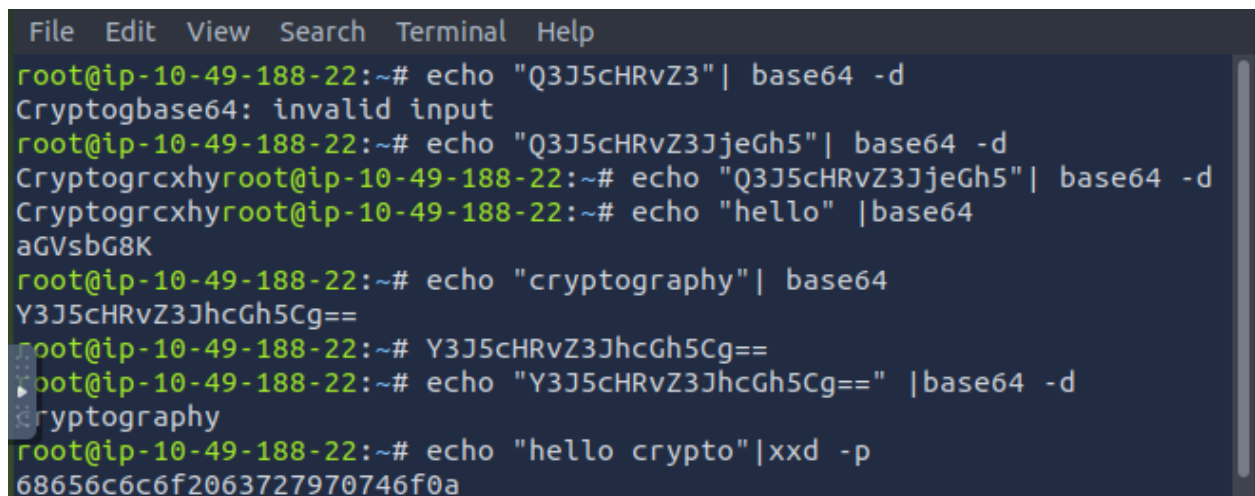
echo 48656c6c6f | xxd -r -p

Text to Hex

echo "Hello" | xxd -p

Expected Output: Hello

OUTPUT:



```
File Edit View Search Terminal Help
root@ip-10-49-188-22:~# echo "Q3J5cHRvZ3" | base64 -d
Cryptogbase64: invalid input
root@ip-10-49-188-22:~# echo "Q3J5cHRvZ3JjeGh5" | base64 -d
Cryptogrcxhyroot@ip-10-49-188-22:~# echo "Q3J5cHRvZ3JjeGh5" | base64 -d
Cryptogrcxhyroot@ip-10-49-188-22:~# echo "hello" | base64
aGVsbG8K
root@ip-10-49-188-22:~# echo "cryptography" | base64
Y3J5cHRvZ3JhcGh5Cg==
root@ip-10-49-188-22:~# Y3J5cHRvZ3JhcGh5Cg==
root@ip-10-49-188-22:~# echo "Y3J5cHRvZ3JhcGh5Cg==" | base64 -d
cryptography
root@ip-10-49-188-22:~# echo "hello crypto" | xxd -p
68656c6c6f2063727970746f0a
```

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

4. Identify the shift value (given or by trial).
5. Use online Caesar decoder or terminal tool.
6. Change the rotation type and get different answers.

Using terminal

```
echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
```

(Shift = 3)

Expected Output: HELLO

OUTPUT:

```
root@ip-10-49-188-22:~# echo "68656c6c6f2063727970746f0a" | xxd -r -p
hello crypto
root@ip-10-49-188-22:~# echo "ceasar ciphers"
ceasar ciphers
root@ip-10-49-188-22:~# echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
HELLO
root@ip-10-49-188-22:~# ECHO "PLAYFAIR CIPHER"
CHO: command not found
root@ip-10-49-188-22:~#
root@ip-10-49-188-22:~# echo "PLAYFAIR CIPHER"
PLAYFAIR CIPHER
```

b) Playfair Cipher

Purpose: To encrypt using alphabets directly

How to Solve

1. In the terminal type nano playfaircipher.py
Playfaircipher.py code
playfair_cipher.py
Usage:
python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"
python3 playfair_cipher.py dec -k "MONARCHY" -t
"BMNDZBXdKYBEJVDMUIXMMOUVIF"

```
import argparse
import re
```

```
def normalize_text(s: str) -> str:
    s = re.sub(r"[^A-Za-z]", "", s).upper()
    s = s.replace("J", "I") # I/J combined
```

```
return s
```

```
def build_square(key: str):  
    key = normalize_text(key)  
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ" # no J  
    seen = set()  
    seq = []  
    for ch in key + alphabet:  
        if ch not in seen:  
            seen.add(ch)  
            seq.append(ch)  
    square = [seq[i:i+5] for i in range(0, 25, 5)]  
    pos = {square[r][c]: (r, c) for r in range(5) for c in range(5)}  
    return square, pos
```

```
def make_digraphs(plain: str, filler: str = "X"):  
    plain = normalize_text(plain)  
    digs = []  
    i = 0  
    while i < len(plain):  
        a = plain[i]  
        b = plain[i+1] if i+1 < len(plain) else ""  
        if b == "":  
            digs.append((a, filler))  
            i += 1  
        elif a == b:  
            digs.append((a, filler))  
            i += 1  
        else:  
            digs.append((a, b))  
            i += 2  
    return digs
```

```
def playfair_encrypt(plain: str, key: str) -> str:  
    square, pos = build_square(key)  
    out = []  
    for a, b in make_digraphs(plain):  
        ra, ca = pos[a]  
        rb, cb = pos[b]  
        if ra == rb: # same row  
            out.append(square[ra][(ca + 1) % 5])  
            out.append(square[rb][(cb + 1) % 5])  
        elif ca == cb: # same col  
            out.append(square[(ra + 1) % 5][ca])  
            out.append(square[(rb + 1) % 5][cb])  
        else: # rectangle  
            out.append(square[ra][cb])  
            out.append(square[rb][ca])  
    return "".join(out)
```

```

def playfair_decrypt(cipher: str, key: str) -> str:
    cipher = normalize_text(cipher)
    if len(cipher) % 2 != 0:
        raise ValueError("Ciphertext length must be even for Playfair.")
    square, pos = build_square(key)
    out = []
    for i in range(0, len(cipher), 2):
        a, b = cipher[i], cipher[i+1]
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb: # same row
            out.append(square[ra][(ca - 1) % 5])
            out.append(square[rb][(cb - 1) % 5])
        elif ca == cb: # same col
            out.append(square[(ra - 1) % 5][ca])
            out.append(square[(rb - 1) % 5][cb])
        else: # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("mode", choices=["enc", "dec"])
    ap.add_argument("-k", "--key", required=True)
    ap.add_argument("-t", "--text", required=True)
    args = ap.parse_args()

    if args.mode == "enc":
        print(playfair_encrypt(args.text, args.key))
    else:
        print(playfair_decrypt(args.text, args.key))

if __name__ == "__main__":
    main()

```

2. Save the file by giving Ctrl + X. It will ask whether to save. Give Y and save the text editor
3. Give the command to run playfair cipher
python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"
4. Give the following command to run decryption
python3 playfair_cipher.py dec -k "MONARCHY" -t "twqe kjl eazq"

OUTPUT

```
File Edit View Search Terminal Help
root@ip-10-49-188-22:~# echo "PLAYFAIR CIPHER"
PLAYFAIR CIPHER
root@ip-10-49-188-22:~# nano playfair_cipher.py
root@ip-10-49-188-22:~# python3 playfair_cipher.py enc -k "MONARCHY" -t
"hide the gold"
BFCKPDFIMPBZ
root@ip-10-49-188-22:~# python3 playfair_cipher.py dec -k "MONARCHY" -t
"twqe kjl eazqx"
OZLGIGECRXSW
root@ip-10-49-188-22:~#
```

c) Vigenère Cipher

Purpose

To encrypt and decrypt a message using a keyword-based substitution cipher.

CODE

```
def vigenere_encrypt(text, key):
    result = ""
    key = key.lower()
    key_index = 0

    for char in text:
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('a')
            start = ord('A') if char.isupper() else ord('a')

            encrypted_char = chr((ord(char) - start + shift) % 26 + start)
            result += encrypted_char

            key_index += 1
        else:
            result += char

    return result

def vigenere_decrypt(text, key):
    result = ""
    key = key.lower()
    key_index = 0
```



```

for char in text:
    if char.isalpha():
        shift = ord(key[key_index % len(key)]) - ord('a')
        start = ord('A') if char.isupper() else ord('a')

        decrypted_char = chr((ord(char) - start - shift) % 26 + start)
        result += decrypted_char

        key_index += 1
    else:
        result += char

return result
message = input("Enter the message: ")

key = int(input("Enter the key: "))

encrypted = caesar_encrypt(message, key)
decrypted = caesar_decrypt(encrypted, key)

print("Encrypted:", encrypted)
print("Decrypted:", decrypted)

```

OUTPUT

```

root@ip-10-49-187-62:~# python3 vignere_cipher.py
Enter the msg :hello h1
entr the key :meet
Encrypted: tpea l1
Decrypted: hello h1
root@ip-10-49-187-62:~# python3 vignere_cipher.py
Enter the msg :meet me at 10.00
entr the key :hello
Encrypted: tpe al ee 10.00
Decrypted: meet me at 10.00
root@ip-10-49-187-62:~# █

```

d) Vernam Cipher

Purpose

To encrypt and decrypt a message using the Vernam Cipher (One-Time Pad) technique based on XOR operation.

Code

```

def vernam_encrypt_alpha(message, key):
    message = message.upper()
    key = key.upper()
    if len(message) != len(key):
        raise ValueError("Key must be same length as message")

```

```

cipher = ""

for m, k in zip(message, key):
    c = (ord(m) - 65) ^ (ord(k) - 65)
    cipher += chr(c + 65)

return cipher

def vernam_decrypt_alpha(cipher, key):
    cipher = cipher.upper()
    key = key.upper()

    message = ""
    for c, k in zip(cipher, key):
        m = (ord(c) - 65) ^ (ord(k) - 65)
        message += chr(m + 65)
    return message

message=input("Enter the plain test : ")
key=input("Enter the key :")
cipher = vernam_encrypt_alpha(message, key)
plain = vernam_decrypt_alpha(cipher, key)
print("Cipher:", cipher)
print("Plain:", plain)

```

OUTPUT

```

root@ip-10-49-187-62:~# python3 vernam.py
enter the plain text: hello
enter the text key :meetw
Encrypted:
Decrypted: hello
root@ip-10-49-187-62:~# python3 vernam.py
enter the plain text: hello
enter the text key :meet
Key must be the same length as the message
Traceback (most recent call last):
  File "vernham.py", line 58, in <module>
    decrypted = vernam_decrypt(encrypted, key)
  File "vernham.py", line 32, in vernam_decrypt
    raise ValueError("Key must be the same length as the cipher")
ValueError: Key must be the same length as the cipher
root@ip-10-49-187-62:~# █

```

Result

Thus, the **Crypto 101** experiment was successfully completed on the **TryHackMe platform**. The experiment helped in understanding **basic cryptographic concepts**, encryption techniques, encoding methods through hands-on practice.